



# Tarea: Tests



## Objetivo:

Consolidar lo trabajado sobre Diseño Orientado a Contratos y avanzar hacia el Diseño Orientado a Pruebas, escribiendo tests sobre la lógica principal del sistema **sin depender de las interfaces de entrada**.

---



## Consigna:

1. Si todavía no lo hiciste: implementá **dos formas de entrada al sistema**:

- Una **interfaz por línea de comandos** (CLI).
- Una **interfaz automática** (desde script o test).



*Nota:* Aunque estas dos formas de entrada son distintas desde lo funcional (son **dos adaptadores**), en términos de código **no deberían implicar dos interfaces distintas**. La lógica del sistema debería estar representada por **una única interfaz o clase abstracta** (por ejemplo, `SistemaDeTransporte`), y ambas entradas deberían usar esa misma interfaz.

2. Escribí **al menos tres tests automatizados**, que:

- No usen entrada/salida (nada de `input()`, `print()`, etc.).
- Prueben directamente clases y métodos del core del sistema.
- Incluyan al menos **dos transportes distintos** (por ejemplo: Auto, Dron).
- Prueben también, si existe, la clase que se encarga de orquestar el viaje.

3. Los tests pueden estar hechos con cualquier framework de testing (`unittest`, `pytest`, `Jest`, `JUnit`, `xUnit`, etc.), pero deben poder ejecutarse **de forma automática y sin intervención del usuario**.

---



## Sugerencias:

- No hace falta testear la CLI ni el script. Solo la lógica de negocio.
- Si tu diseño está bien desacoplado, deberías poder testear todo sin tocar las interfaces de entrada.
- Si ya tenías buena separación, ¡esta parte debería resultarte fácil!

---

🗨 Preguntas para pensar:

- ¿Qué parte fue más fácil de testear? ¿Y cuál más difícil?
- ¿Qué te facilitó el hecho de haber separado antes la lógica de las interfaces?
- ¿Qué hubieras hecho distinto si hubieras tenido que testear desde el inicio?